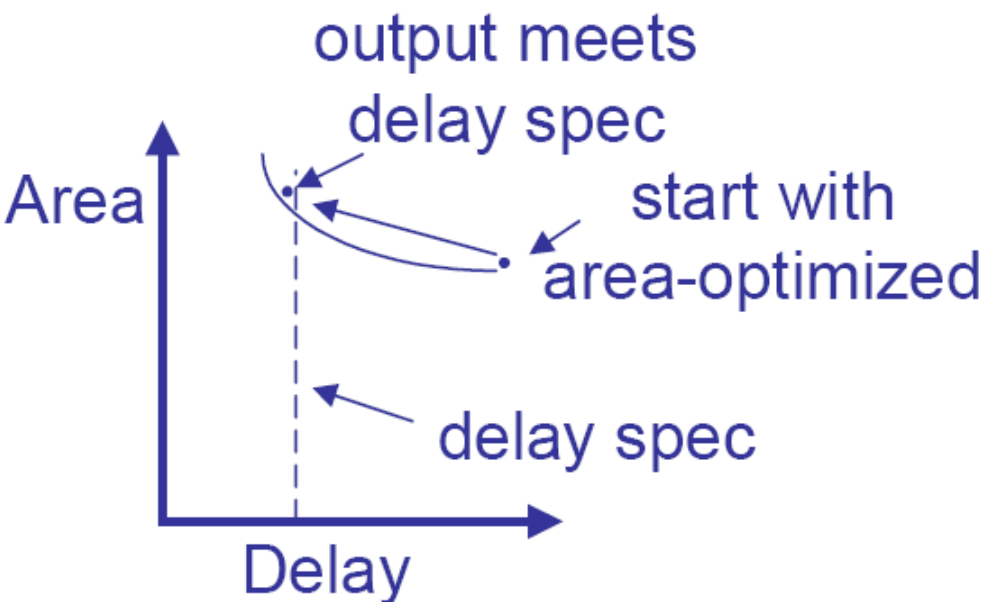
ALU HW1:
FLP and FXP Adder and Multiplier

ALU HW1 Outlines

- 32-bit signed fixed-point (FXP) adder
  - synthesis constraint: delay-opt, area-opt, in-between
- 32-bit floating-point (FLP) adder
  - non-pipelined (single cycle)
  - 7 pipelined stages
    - ✓ unpack, align, FXP add, convert to magnitude, noralize, round, unpack
  - 4 pipelined stages
    - ✓ unpack+align
    - ✓ FXP add+convert_from_sign_to_magnitude
    - ✓ normalize
    - ✓ round+pack
- 32x32 FXP multiplier (output 32 MSB)
- 32-bit FLP multiplier
  - non-pipelined
  - 3 pipelined stages (unpack, FXP multiply, normalize+round+pack)
- Lab slides for EDA tools
  - vcs, Verdi, Design Compiler

FXP_adder (32-b signed)

- fixed-point (FXP) adder with 32-bit signed inputs
  - Synopsys DC synthesis with three different constraints
    - ✓ delay-optimized
    - ✓ area-optimized
    - ✓ in-between



- synthesis report
  - area (combinational + sequential logic)
  - delay
  - power

Overview of IEEE 754 FLP Standard Formats

Some features of the ANSI/IEEE standard floating-point number representation formats.

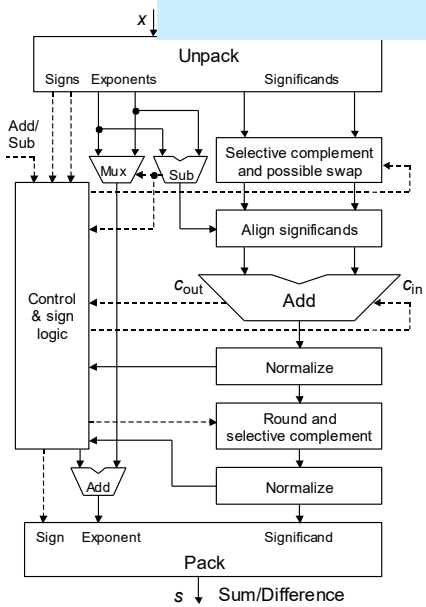
Feature	Single / Short	Double / Long
Word width (bits)	32	64
Significand bits	23 + 1 hidden	52 + 1 hidden
Significand range	$[1, 2 - 2^{-23}]$	$[1, 2 - 2^{-52}]$
Exponent bits	8	11
Exponent bias	127	1023
Zero (± 0)	$e + bias = 0$, $f = 0$	$e + bias = 0$, $f = 0$
Denormal	$e + bias = 0$, $f \neq 0$	$e + bias = 0$, $f \neq 0$
	represents $\pm 0.f \times 2^{-126}$	represents $\pm 0.f \times 2^{-1022}$
Infinity ($\pm\infty$)	$e + bias = 255$, $f = 0$	$e + bias = 2047$, $f = 0$
Not-a-number (NaN)	$e + bias = 255$, $f \neq 0$	$e + bias = 2047$, $f \neq 0$
Ordinary number	$e + bias \in [1, 254]$	$e + bias \in [1, 2046]$
	$e \in [-126, 127]$	$e \in [-1022, 1023]$
	represents $1.f \times 2^e$	represents $1.f \times 2^e$
min	$2^{-126} \cong 1.2 \times 10^{-38}$	$2^{-1022} \cong 2.2 \times 10^{-308}$
max	$\cong 2^{128} \cong 3.4 \times 10^{38}$	$\cong 2^{1024} \cong 1.8 \times 10^{308}$

FLP Add/sub - Alignment

Addition: $x + y = (\pm\, s1 \times b^{\,e1}) + (\pm\, s2 \times b^{\,e2})$

Assume $e1 \geq e2$; *alignment* ***right*** *shift* (*pre-shift*) of y is needed if $e1 > e2$

$$(\pm\, s1 \times b^{e1}) + (\pm\, s2 \times b^{e2}) = (\pm\, s1 \times b^{e1}) + (\pm\, s2 / b^{e1-e2}) \times b^{e1}$$
$$= (\pm\, s1 \pm s2 / b^{e1-e2}) \times b^{e1} = \pm\, s \times b^{e}$$



Example: Numbers to be added:

$$x = 2^5 \times 1.00101101$$
$$y = 2^1 \times 1.11101101$$

Operand with smaller exponent to be preshifted

Operands after alignment shift:

$$x = 2^5 \times 1.00101101$$
$$y = 2^5 \times 0.000111101101$$

Extra bits to be rounded off

Result of addition:

$$s = 2^5 \times 1.01001011\overbrace{1101}$$
$$s = 2^5 \times 1.01001100$$

Rounded sum

(1) Pre-alignment;
(2) Fixed-point arithmetic;
(3) Post-normalization;
(4) Rounding
(5) Possible normalization again

normalization for add/sub (two cases)

- **Case-I (Far Case): abs(e1-e2) >= 2,**
  - **normalization of shift at most one bit**

```
       1.10011111
       0.00101011
ADD    ----------
       1.11001010
```

Exponent difference is large (3 in this case)
No normalization is needed

$$1 \le s_1 < 2,\ 1 \le s_2 < 2$$

$$1/4 \le s_2 \times 2^{-2} < 1/2$$

$$\underbrace{\frac{1}{2}}_{0.1_2} < s_1 \pm s_2 \times 2^{-2} < 2\frac{1}{2}$$

```
       1.1001111
       0.0110110
ADD    ---------
      10.0000101
NORM   1.00000101
```

Significand is larger than 2
normalization of right shift 1-bit is needed

Quiz: Give an example that normalization requires 1-bit left shift

- **Case-II (Near Case): abs(e1-e2) <=1,**
  - **normalization of arbitrary shift bits**

$$1 \le s_1 < 2,\ 1 \le s_2 < 2$$

$$1/2 \le s_2 \times 2^{-1} < 1,$$

$$0 < s_1 \pm s_2 \times 2^{-1} < 3$$

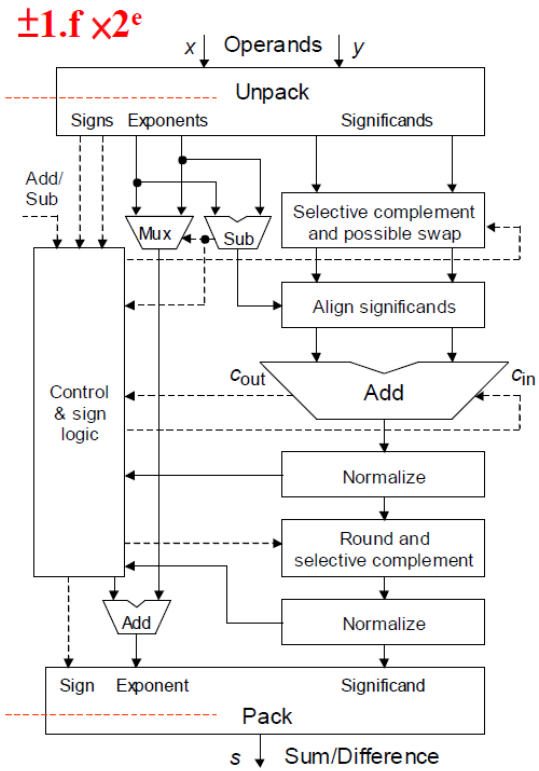
$$-1 < s_1 \pm s_2 < 4$$

```
       1.1001111
       1.1001010
SUB    ---------
       0.0000101
NORM   1.0100000
```

Significand has many leading zeros
normalization of left shift many bits is needed

FLP_adder

- D=A+B with IEEE 32-bit floating-point (FLP) inputs
  - execution within a single cycle
- composed of the following steps
  - unpack
  - alignment
  - FXP add/sub
  - conversion to sign-magnitude
  - normalization
  - rounding
  - pack
- guard bits for FXP add/sub
  - at least 3 integer bits to cover range of signed results
  - smaller number is ignored if $|e_1-e_2| > 25$
    - ✓ see next slide

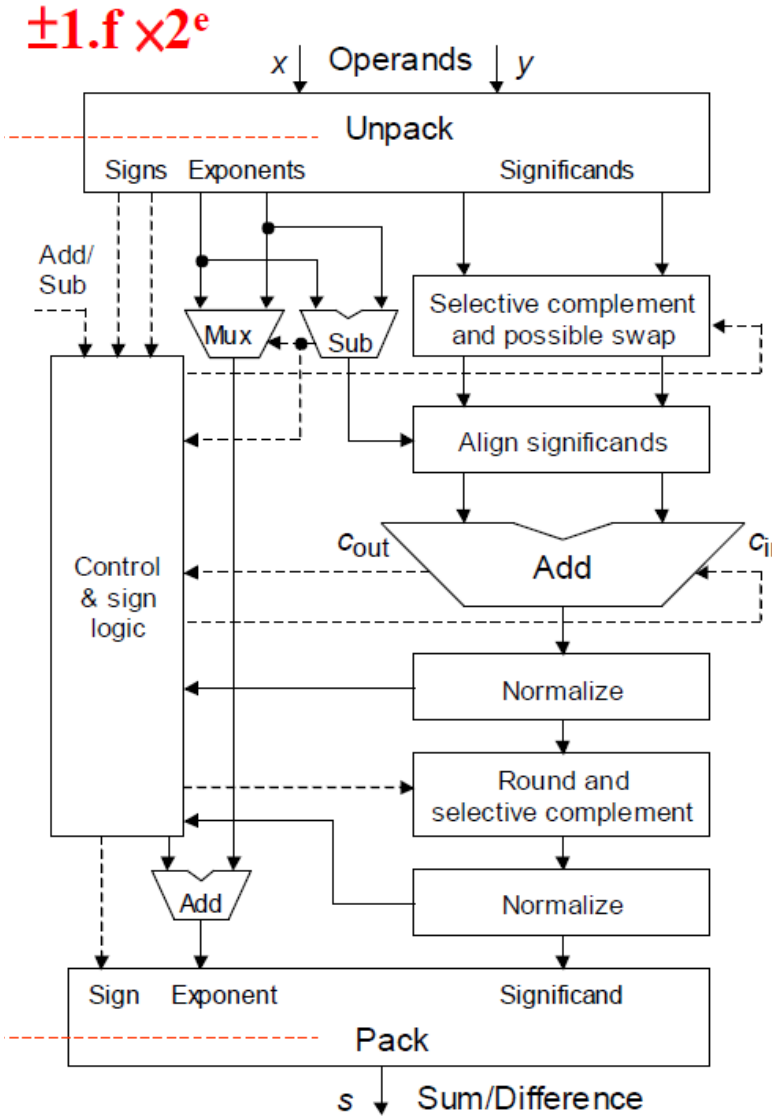


alignment only for $-25 <= e_a - e_b <= 25$

- denote (exponent, *signed* mantissa) as
  - A: (e_a, m_a), B: (e_b, m_b)
  - $|m_a| < 2$ $(-2 < m_a <= 1$, or $1 <= m_a < 2)$, similarly for m_b
  - m_a, m_b could be represented as Q2.23 (xx.xx…x)
- assume $e_a > e_b$, exponent difference $ed = e_a - e_b$
  - If ed >= 26, and alignment-right-shifted $m_b \rightarrow m'_b$
  - $|m'_b| < 2^{-25}$ (because $|m_b| < 2$)
  - after possible 1-bit normalization left shift, the impact on add result is $< 2^{-24}$, which could be ignored after rounding
- similarly for $e_a < e_b$
- thus, could consider only $-25 <= e_a - e_b <= 25$
  - alignment with ed = 26 or more could be ignored
- Note for the possible 1-bit left shift normalization of add result, which could amplify error by a factor of 2

when to convert signed result to sign-magnitude?

- FLP adder of right figure
  - signed fixed-pt. add/sub result x is converted to sign-magnitude **after** normalization and rounding
  - *need to find leading 1 or 0*
    - ✓ *positive* (00…001xx…x), or
    - ✓ negative (11…110xx…x)
  - *2 possible cases for negative*
    - ✓ x=**10**.xx…x (-2 <x< -1), or
    - ✓ x=**11**.000…0 (x = -1)
  - more complicate
- suggested for Homework
  - signed. result x is converted to magnitude **before** normalization
  - only need to find leading 1 for |x|
    - ✓ 01.xxx…x (1 <= |x| < 2)



suggested FLP add/sub architecture

- unpack
  - extract sign, exponent, mantissa, and operand with smaller exponent (for subsequent alignment)
  - convert mantissa to Q2.25 signed format (why?)
- alignment
- signed add/sub
- signed-to-sign/magnitude conversion
- normalization
  - find leading 1 and perform shift
  - adjust exponent
- rounding (and possibly re-normalization)
  - another normalization might be required in case 2
- pack

alignment and normalization

- after converting mantissa to signed => Q2.23
- consider range increase => Q3.23
- near case (ed =1 or 0)
  - alignment right shift <= 1
  - normalization shift <= 24
    - ✓ leading 1 might be in position 2^{-24}
  - Q3.24 is enough for the at most 1-bit alignment right shift
- far case (ed =2 or more)
  - alignment right shift <= 25
  - normalization shift <=1
    - ✓ magntidue of addition result >= ½
  - Q3.25 is enough for at most 1-bit normalization left shift and round bit
- Thus, Q3.25 is enough for *round-to-nearest*

Pipeline
(example: pipelined Multiply-Add (MAD))

- two pipelined stages with two always blocks

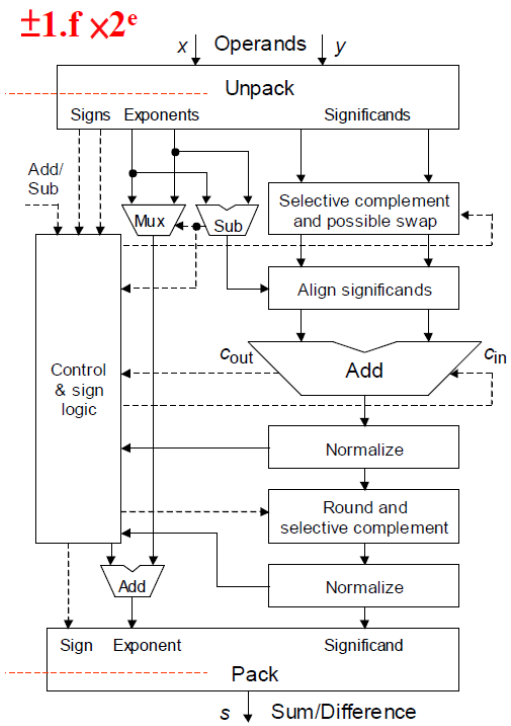
```
module MAD (input [n-1] a, b, input [2n-1:0] c,  output reg [2*n-1:0] mad);
parameter n=16;
wire clk;
reg [2*n-1:0] p, c_tmp;

// 1st  pipeline stage that performs multiplication
always @ (posedge clk) begin
 p <= a*b;          // register to store the product a*b
c_tmp <= c;      // register to store c, to be used in 2nd stage
end

// 2nd pipeline stage that performs addition
always @ (posedge clk) begin
   mad <= p + c_tmp;
end
endmodule
```

FLP_adder_pipe_7

- pipelined floating-point adder
- each following step is a stage
  - unpack
  - align
  - FXP add
  - convert to sign-magnitude
  - normalize
  - round
  - pack



- one **always @ posedge** clk block for each stage
  - RHS are stage inputs, LHS are stage outputs
- latency vs. throughput
- identify the delay of each pipelined stage

unpack

- given (s1, e1, f1), (s2, e2, f2)
- determine bit-width for signed mantissa Q3.25
  - consider FLP add results and normalization/rounding
- perform possible swap so that ea >= eb
  - ea = (e1>e2) ? e1-bias : e2-bia;
  - eb = (e1>e2)? e2-bias : e1-bias;
- calculate exponent difference e_diff = ea-eb
- calculate signed mantissa from sign bits sa, sb, and fractional bits fa, fb
  - wire [1:23] fa, fb; // fractional bits of a, b
  - signed [27:0] ma = {2'b00, 1'b1, fa, 2'b00}; // Q3.25
  - signed [27:0] mb = {2'b00, 1'b1, fb, 2'b00};
  - ma_signed = (sa)? ~ma+1 : ma;
  - mb_signed = (sb)? ~mb+1 : mb;

align

- given mb_signed, e_diff
- right-shift mb_signed to mb_aligned
  - mb_aligned = mb_signed >>> e_diff;
    - ✓ **arithmetic** right shift for signed numbers

FXP add

- given ma_signed, mb_aligned
- perform signed = fixed-point add d = a + b
  - md_signed = ma_signed + mb_aligned
  - 0 <= |md_signed| < 4

convert to sign-magnitude

- given md_signed of Q3.25
- convert signed result to sign-magnitude
  - md = md_signed[MSB] ? ~md_signed+1 : md_signed;
  - sd = md_signed[MSB] ? 1 : 0; // MSB=27 for Q3.25
  - 0<= md < 4

normalize

- given magnitude md of format Q3.25
  - md = 0xx.xxx…x
- find # of 0's before leading 1 => zero_count
  - shift_left = zero_count -2;
    - ✓ e.g. 01x.xxxx…x => shift_left = -1 // right-shift 1 bit
    - ✓ e.g., 001.xxxx…x => shift_left = 0
    - ✓ e.g., 000.1xxx…x => shift_left = 1
    - ✓ e.g., 000.01xx…x => shift_left = 2
    - ✓ …
- perform normalization shift
  - md_norm = md << shift_left; // **logical** shift for unsigned
  - 1 <= md_norm < 2
- calculate exponent of d = a + b
  - ed = ea – shift_left

round

- given 1 <= md_norm <2 of Q3.25
- round md_norm based on md_norm[1]
  - wire [27:0] md_norm = 001.xxx….x // Q3.25
  - md_round = md_norm[1] ? md_norm+28'd4 : md_norm;
  - wire [23:0] md_final = md_round[25:2]; // Q1.23
- for simplicity, only consider *round-to-nearest*
  - i.e., in mid-way case, always add 1 to LSB
- in most cases, 1 <= md_round < 2
  - i.e., md_round = 001.xxx…x
- but it is possible that md_round = 2
  - i.e., md_round = 010.000…0
  - re-normalization with 1-bit right-shift required

pack

- given sd, ed, md_final
- calculate biased exponent
  - ed_final = ed + bias
- extract fractional bits fd of md_final of Q1.23
  - wire [22:0] fd = md_final[[22:0];

FLP_adder_pipe_4

- fixed-point adder (FXP_adder)
  - synthesis with three different constraints
    - ✓ delay-optimized, area-optimized, and in-between
- non-pipelined floating-point adder (FLP_adder)
  - unpack + align
  - FXP add + convert to sign-magnitude
  - normalize
  - round + pack
- Do you have better pipeline strategy?
  - more balanced delay in each stage
- placement and routing (P&R) of FLP_adder_pipe_4
  - mark the four pipelined stages on the layout view

Floating-Point Multipliers

$$(\pm\, s1 \times b^{e1}) \times (\pm\, s2 \times b^{e2}) \;=\; (\pm\, s1 \times s2) \times b^{e1+e2}$$

$s1 \times s2 \in [1, 4)$: may need postshifting

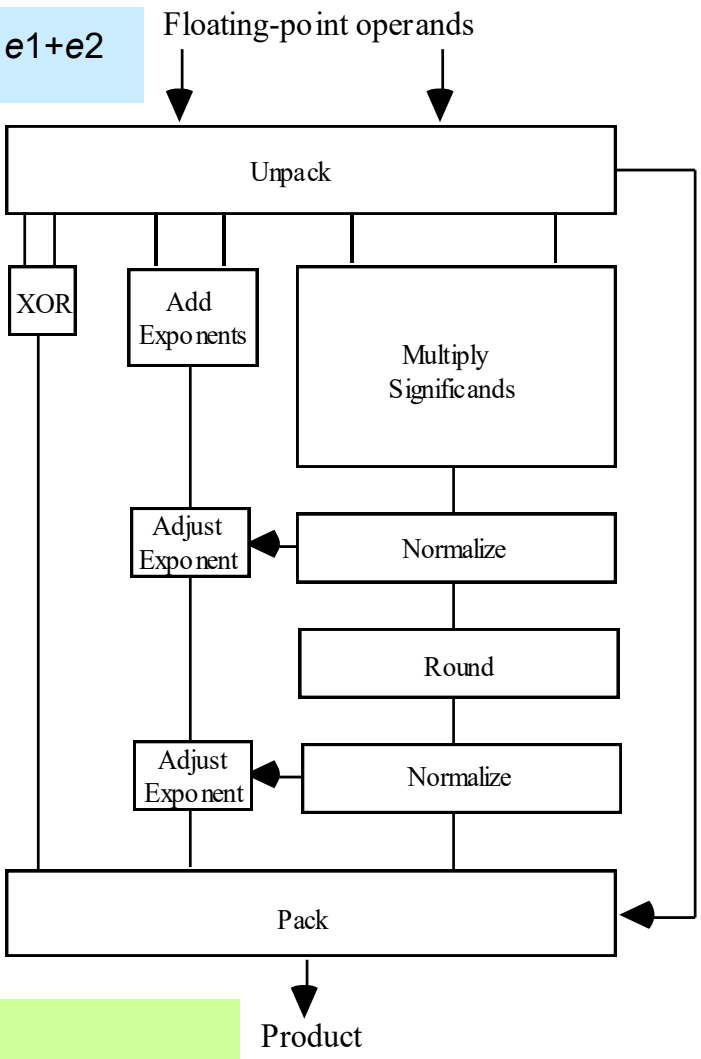
Overflow or underflow can occur during multiplication or normalization

Speed considerations

Many multipliers produce the lower half of the product (rounding info) early

Need for normalizing right-shift is known at or near the end

Hence, rounding can be integrated in the generation of the upper half, by producing two versions of these bits



Block diagram of a floating-point multiplier.

FXP_multiplier (32-b MSB output)

- multiplier with 32-bit fixed-point signed inputs
  - 64-bit output
  - overflow/underflow occurs more often than FLP add/sub
    - ✓ due to the addition of exponents which is easily over the maximum/minimum exponent boundary
  - DC synthesis with three different constraints
    - ✓ delay-optimized
    - ✓ area-optimized
    - ✓ in-between
- synthesis report
  - area
  - delay
  - power (dynamic + leakage)

Floating-Point Multipliers

$$(\pm\, s1 \times b^{e1}) \times (\pm\, s2 \times b^{e2}) \;=\; (\pm\, s1 \times s2) \times b^{e1+e2}$$

$s1 \times s2 \in [1, 4)$: may need post-shifting

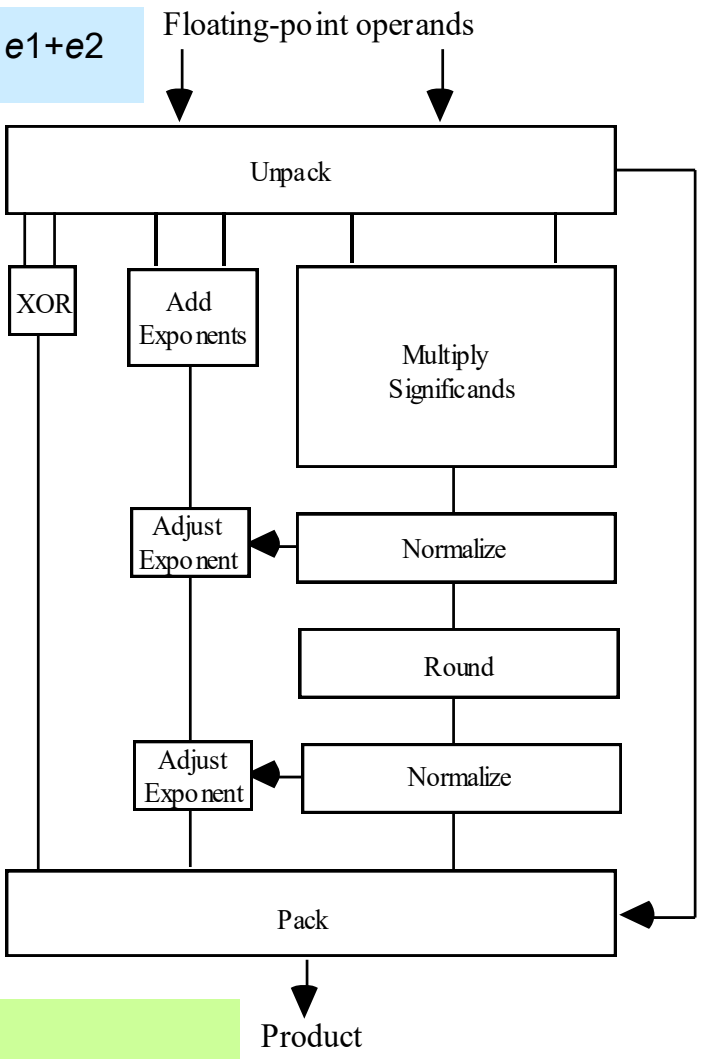
Overflow or underflow can occur during multiplication or normalization

Speed considerations

Many multipliers produce the lower half of the product (rounding info) early

Need for normalizing right-shift is known at or near the end

Hence, rounding can be integrated in the generation of the upper half, by producing two versions of these bits



Block diagram of a floating-point multiplier.

FLP_multiplier

- IEEE 32-bit floating-point multiplier
- composed of the following operations
  - input data unpacking
  - direct fixed-point (FXP) unsigned multiplication of mantissa
  - simple normalization
  - rounding
  - output result packing
- no alignment needed
- direct FXP unsigned multiplication of mantissa magnitude
  - *no need to convert mantissa to signed format*
- normalization is possible right-shift at most 1-bit

FLP_multiplier_pipe_3

- multiplier with 32-bit fixed-point signed inputs
  - no alignment needed
  - simpler normalization than FLP addition
- composed of the following operations
  - Unpacking (1st sage)
    - ✓ no alignment needed
  - FXP multiplication (2nd stage)
    - ✓ no signed conversion needed
  - normalization, rounding, packing (3th stage)
    - ✓ normalization is much simpler than FLP addition

comparison table

- table of summary

		Area (um²)			Delay	Latency	Power (W)		
		CL	SL	Total	(ns)	(ns)	dynamic	leakage	total
FXP_adder	area								
	mid								
	delay								
FLP_adder	area								
	mid								
	delay								
FLP_adder_pipe_7	area								
	mid								
	delay								
FLP_adder_pipe_4	area								
	mid								
	delay								
FXP_multiplier	area								
	mid								
	delay								
FLP_multiplier	area								
	mid								
	delay								
FLP_multiplier_pipe_3	area								
	mid								
	delay								